SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 10%

Duration :60 Mins
On Class
QUESTIONS: 1 **Total Marks:50**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I Ranjitha. R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Language Processors: Compiler vs Interpreter

CSA1408 - Compiler Design

SUPERVISOR:
Dr. G. Michael
Professor
Department of CSE

BY:
Name : Ranjitha .R
Reg No : 192421285

Language Processor

- A language processor is software that converts a high-level programming language into machine language.
- It helps the computer understand and execute programs.

Types of Language Processors

- Compiler
- Interpreter
- Assembler

Examples of High-Level Languages:

- C
- C++
- Java
- Python

Compiler

- A compiler translates the **entire program** into machine code before execution.

Features :

- Converts the whole program at once.
- Generates an executable file.
- Executes programs faster.
- Shows all errors after compilation.

Examples

- C Compiler (GCC)
- C++ Compiler (G++)

Compiler

Source Code
↓
Lexical Analysis
↓
Syntax Analysis
↓
Semantic Analysis
↓
Intermediate Code Generation
↓
Code Optimization
↓
Target Code Generation
↓
Executable Program

compiler optimization techniques used to make programs run faster, use less memory, and improve efficiency without changing the program's output

Important Optimizations

- Constant Folding
- Dead Code Elimination
- Loop Optimization
- Function Inlining
- Register Allocation
- Common Subexpression Elimination

Interpreter

- An interpreter translates and executes the program **one line at a time**.

Features

- No executable file is created.
- Executes line by line.
- Stops when an error is found.
- Easier for debugging but slower than a compiler.

Examples

- Python Interpreter
- JavaScript Engine

Interpreter

Source Code
↓
Tokenizer
↓
Parser
↓
Abstract Syntax Tree (AST)
↓
Runtime Execution
↓
Output

Characteristics

- Immediate execution without generating an executable.
- Runtime type checking.
- Dynamic memory allocation.
- Interactive debugging support.
- High flexibility for scripting and rapid development.

Performance Challenges

- Repeated parsing.
- Higher CPU overhead.
- Slower execution for computationally intensive applications.

Compiler vs Interpreter

Compiler	Interpreter
Translates the whole program	Translates one line at a time
Faster execution	Slower execution
Creates an executable file	No executable file
Reports all errors together	Stops at the first error
Example: GCC, G++	Example: Python, JavaScript

Conclusion

- Compiler is faster and suitable for large applications.
- Interpreter is easier to test and debug, making it ideal for scripting and rapid development.
- Both language processors convert high-level code into a form the computer can execute.

Engineer to Excel

GEO TAG PHOTO

THANK YOU